

Design and Implementation of a Basic 16-bit RISC Processor

on FPGA — Boolean Board (Xilinx Spartan-7)

Course	FPGA Based Design
Platform	Boolean Board — Xilinx XC7S50 Spartan-7
Tool	Xilinx Vivado 2020.x / 2023.x
Language	Verilog HDL
Architecture	8-bit data, 16-bit fixed instruction, 8 registers, sequential execution
Difficulty	Advanced — Multi-module Processor Design

1. Introduction

A Reduced Instruction Set Computer (RISC) processor is the foundational architecture behind virtually every modern processor. The defining principles of RISC design — fixed-width instructions, load/store memory model, a small uniform register file, and single-cycle or pipelined execution — make it an ideal subject for FPGA implementation. Building a processor from scratch requires integrating all major concepts of digital design: instruction set architecture, datapath design, control logic, register file organisation, ALU construction, and memory interfacing.

This assignment requires students to design, simulate, and deploy a basic 16-bit RISC processor on the Boolean Board (Xilinx XC7S50 Spartan-7). The processor implements a custom ISA supporting arithmetic, comparison, and memory operations over an 8-bit datapath. All instructions are encoded as fixed 16-bit words. Execution is sequential — one instruction completes before the next is fetched. No branch or jump instructions are required.

This is the most complex assignment in the series, requiring students to integrate multiple independently designed modules — Program Counter, Instruction Memory, Register File, ALU, Multiplier, Data Memory, and Control Unit — into a single working processor top module, verify it with a mandatory test program, and demonstrate it on hardware.

2. Objective

The primary objectives of this laboratory assignment are:

- Design a custom 16-bit fixed-width ISA supporting arithmetic, comparison, and load/store memory operations.
- Implement a multi-module RISC processor datapath in Verilog HDL: PC, instruction memory, register file, ALU, multiplier, data memory, and control unit.
- Design a synchronous control unit (decoder) that generates all datapath control signals from the 4-bit opcode.
- Execute the mandatory test program correctly and verify results against expected register and memory values.
- Synthesise, implement, and demonstrate the processor on the Boolean Board using switches, LEDs, and/or 7-segment display.
- Produce simulation waveforms showing correct fetch-decode-execute behaviour for each instruction in the test program.
- Document the design with a block diagram, ISA encoding table, datapath description, and implementation report.

3. Processor Specifications

The processor must conform to the following architectural parameters. These are fixed constraints and must not be modified.

Parameter	Specification
Data width	8-bit (all registers, ALU operands, and memory data are 8-bit)
Instruction width	16-bit fixed width (all instructions occupy exactly 16 bits)
General-purpose registers	8 registers — R0 through R7, each 8-bit wide
Program Counter (PC)	Increments by 1 after each instruction; no branch/jump support
Instruction memory	ROM / BRAM — preloaded with the program; addressed by PC
Data memory	RAM / BRAM — 512 locations, each 8-bit wide (9-bit address)
Execution model	Sequential, non-pipelined — one instruction completes before the next is fetched
Supported operations	NOP, ADD, SUB, MUL, CMP, MOV, LOAD, STORE, HALT
Comparison flags	Z (zero/equal), G (greater), L (less) — updated by CMP instruction only

4. Instruction Set Architecture (ISA)

4.1 Instruction Formats

All instructions are exactly 16 bits wide. Two instruction formats are defined based on the operation type:

R-Type — Arithmetic, Logic, and Comparison Instructions

[15:12] opcode	[11:9] rd	[8:6] rs1	[5:3] rs2	[2:0] unused
4 bits	3 bits — destination register	3 bits — source register 1	3 bits — source register 2	3 bits — set to 000

Used by: ADD, SUB, MUL, CMP, MOV, NOP.

M-Type — Memory Instructions

[15:12] opcode	[11:9] rd / rs	[8:0] address
4 bits	3 bits — destination (LOAD) or source (STORE)	9 bits — data memory address (range 0 to 511)

Used by: LOAD, STORE.

4.2 Opcode Encoding and Instruction Semantics

Opcode	Mnemonic	Operation	Description
0000	NOP	No operation	The processor advances PC by 1. No register or memory is modified. Used for pipeline alignment or intentional delay.
0001	ADD	$Rd \leftarrow Rs1 + Rs2$	8-bit unsigned addition. Result written to destination register Rd. Carries beyond 8 bits are discarded.
0010	SUB	$Rd \leftarrow Rs1 - Rs2$	8-bit unsigned subtraction. Result written to Rd. Borrows beyond 8 bits are discarded.
0011	MUL	$Rd \leftarrow Rs1 \times Rs2$	8-bit multiplication. The lower 8 bits of the 16-bit product are written to Rd. A dedicated multiplier module is required.
0100	CMP	Set Z, G, L flags	Compares Rs1 and Rs2. Sets Z=1 if $Rs1 == Rs2$, G=1 if $Rs1 > Rs2$, L=1 if $Rs1 < Rs2$. Rd field is unused. No register is written.

Opcode	Mnemonic	Operation	Description
0101	MOV	$Rd \leftarrow Rs1$	Copies the value of register Rs1 into register Rd. Rs2 field is unused.
0110	LOAD	$Rd \leftarrow MEM[address]$	Reads the 8-bit value at the 9-bit data memory address and writes it into register Rd.
0111	STORE	$MEM[address] \leftarrow Rs$	Writes the 8-bit value in register Rs (encoded in bits [11:9]) to the 9-bit data memory address.
1111	HALT	Freeze PC; stop fetch	Stops processor execution. PC is frozen. The processor remains in HALT state until reset is asserted.

4.3 Comparison Flags

The CMP instruction updates three 1-bit flag registers. These flags are not directly accessible via instructions in this ISA but must be observable on FPGA LEDs for demonstration purposes.

Flag	Condition	Description
Z	$Rs1 == Rs2$	Zero flag — set HIGH when the two compared operands are equal.
G	$Rs1 > Rs2$	Greater flag — set HIGH when Rs1 is strictly greater than Rs2.
L	$Rs1 < Rs2$	Less flag — set HIGH when Rs1 is strictly less than Rs2.

Only one flag is HIGH at a time. All three flags are cleared to zero at reset and updated exclusively by the CMP instruction.

5. Instruction Execution Flow

Every instruction — regardless of type — must pass through the following six-stage sequential execution flow. Each stage occupies one clock cycle in the non-pipelined implementation:

Stage	Name	Action
1	Fetch	$IR \leftarrow InstrMem[PC]$. The 16-bit instruction at the current PC address is loaded into the Instruction Register.
2	Decode	The Control Unit decodes bits [15:12] (opcode) and generates RegWrite, MemRead, MemWrite, ALUSrc, ALUOp, and WBSel control signals.

Stage	Name	Action
3	Read Operands	The Register File outputs the values of Rs1 (bits [8:6]) and Rs2 (bits [5:3]). For M-type instructions, the 9-bit address (bits [8:0]) is extracted.
4	Execute	The ALU or Multiplier performs the required operation. For CMP, flags Z, G, L are updated. For MOV, the Rs1 value is passed through unchanged.
5	Memory Access	For LOAD: DataMem[address] is read. For STORE: Rs value is written to DataMem[address]. All other instructions skip this stage.
6	Write Back	The result (from ALU, Multiplier, or Data Memory) is written into the destination register Rd. CMP and STORE do not write to any register.

After Write Back, PC is incremented by 1 ($PC \leq PC + 1$) and the next instruction is fetched. The HALT instruction freezes PC and prevents further fetching.

6. Required Design Modules

The processor must be constructed from the following nine mandatory modules. Each module must be designed, simulated independently, and then integrated into the processor top module.

Module	Description and Interface Requirements
Program Counter (PC)	8-bit or wider counter. Increments by 1 on every clock edge during normal execution. Freezes on HALT. Resets to 0 on rst.
Instruction Memory	Read-only memory (ROM or initialised BRAM). Addressed by PC. Outputs a 16-bit instruction word. Preloaded with the test program using \$readmemb or localparams.
Instruction Register	16-bit register that latches the instruction fetched from Instruction Memory on each clock cycle. Holds the instruction stable throughout Decode, Execute, and Write Back.
Register File	8 x 8-bit synchronous register array (R0–R7). Supports simultaneous read of two source registers and synchronous write to one destination register. R0 may optionally be hardwired to zero.
ALU	Combinational unit supporting ADD, SUB, and CMP. Takes two 8-bit operands and a 2-bit ALUOp control signal. Outputs an 8-bit result and updates Z, G, L flags for CMP.
Multiplier	Dedicated 8-bit x 8-bit combinational or sequential multiplier. Outputs the lower 8 bits of the 16-bit product to the register file write-back bus.
Data Memory	512 x 8-bit synchronous RAM (BRAM or register array). 9-bit address input. Separate read (LOAD) and write (STORE) enable signals. Initialised with test data.

Module	Description and Interface Requirements
Control Unit (Decoder)	Combinational decoder. Input: 4-bit opcode. Outputs: RegWrite, MemRead, MemWrite, ALUSrc, ALUOp[1:0], WBSel[1:0], MulEn, HaltEn. Generates all datapath control signals.
Processor Top Module	Top-level module that instantiates and interconnects all sub-modules. Exposes: clk, rst, LED[7:0], flag_Z, flag_G, flag_L, and optional debug ports (reg_sel, reg_out).

7. Port Specification

7.1 Top-Level Inputs

Signal	Width	Description
clk	1-bit	System clock — positive edge triggered. All state transitions occur on the rising edge.
rst	1-bit	Synchronous active-HIGH reset — clears PC, IR, all registers, flags, and returns processor to Fetch stage.
SW[2:0]	3-bit	Register select (optional) — used on FPGA to select which register (R0–R7) to display on LEDs or 7-segment display.

7.2 Top-Level Outputs

Signal	Width	Description
LED[7:0]	8-bit	Register or memory output — displays the 8-bit value of the selected register (via SW[2:0]) or a fixed register after HALT.
flag_Z	1-bit	Zero flag output — driven HIGH when Z flag is set after CMP. Mapped to a dedicated LED.
flag_G	1-bit	Greater flag output — driven HIGH when G flag is set after CMP. Mapped to a dedicated LED.
flag_L	1-bit	Less flag output — driven HIGH when L flag is set after CMP. Mapped to a dedicated LED.

7.3 Verilog Module Interface (Reference)

```
module risc_processor (
    input          clk,
```

```

input      rst,
input  [2:0] SW,          // Register select for display
output [7:0] LED,         // Selected register value
output     flag_Z,        // CMP equal flag
output     flag_G,        // CMP greater flag
output     flag_L         // CMP less flag
);
// Internal modules: PC, InstrMem, IR, RegFile,
//                  ALU, Multiplier, DataMem, ControlUnit
// ... your implementation here ...
endmodule

```

8. Mandatory Test Program

The following program must be loaded into instruction memory and executed correctly. Simulation waveforms and FPGA results must both be verified against the expected outputs listed below. No marks will be awarded for FPGA implementation without a passing test program.

8.1 Initial Data Memory State

Address	Value (decimal)	Value (8-bit binary)
MEM[10]	6	0000_0110
MEM[11]	4	0000_0100

8.2 Instruction Sequence

PC	Instruction	16-bit Encoding	Operation
0	LOAD R1, 10	0110_001_000001010	R1 <= MEM[10] = 6
1	LOAD R2, 11	0110_010_000001011	R2 <= MEM[11] = 4
2	ADD R3, R1, R2	0001_011_001_010_000	R3 <= R1 + R2 = 10
3	SUB R4, R1, R2	0010_100_001_010_000	R4 <= R1 - R2 = 2
4	MUL R5, R3, R2	0011_101_011_010_000	R5 <= R3 x R2 = 40
5	STORE R5, 20	0111_101_000010100	MEM[20] <= R5 = 40
6	CMP R3, R4	0100_000_011_100_000	R3>R4 => G=1, Z=0, L=0
7	HALT	1111_000_000_000_000	Freeze PC; stop execution

8.3 Expected Results After HALT

Register / Location	Expected Value	Source Instruction	Verification
R1	6	LOAD R1, 10	LED shows 0000_0110
R2	4	LOAD R2, 11	LED shows 0000_0100
R3	10	ADD R3, R1, R2	LED shows 0000_1010
R4	2	SUB R4, R1, R2	LED shows 0000_0010
R5	40	MUL R5, R3, R2	LED shows 0010_1000
MEM[20]	40	STORE R5, 20	Read back via LOAD
flag_G	1 (R3 > R4)	CMP R3, R4	Dedicated LED = HIGH
flag_Z	0	CMP R3, R4	Dedicated LED = LOW
flag_L	0	CMP R3, R4	Dedicated LED = LOW

9. Design Constraints

The following constraints are mandatory and must be strictly observed. Designs that violate these constraints will be penalised.

- All instructions must be exactly 16 bits wide. No variable-length encoding is permitted.
- No branch or jump instructions are required or permitted in this ISA.
- Instruction memory and data memory must be separate modules — a unified memory is not acceptable.
- All arithmetic and logic operations (ADD, SUB, MUL, CMP, MOV) must use register operands only. Immediate operands are not part of the base ISA.
- The MUL instruction must use a dedicated Multiplier module — the ALU must not perform multiplication.
- The register file must support simultaneous read of two registers in a single clock cycle.
- All registers (PC, IR, Register File, flags) must be synchronously reset on the active-HIGH rst signal.
- The processor must remain in the HALT state indefinitely until rst is asserted.
- All module port names and widths must match those specified in Section 7 for testbench compatibility.

End of Problem Statement